

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

CHANDANA J KUMAR

(1BM24CS079)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by CHANDANA J KUMAR(1BM24CS079), who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Kanchana M Dixit
Associate Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph. Leetcode 207: Course Schedule	5-7
2	Implement Johnson Trotter algorithm to generate permutations.	8-10
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	11-13
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken. Leetcode 190: Reverse Bits	14-17
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	18-20
6	Implement 0/1 Knapsack problem using dynamic programming. Leetcode 70.:Climbing Stairs	21-23
7	Implement All Pair Shortest paths problem using Floyd's algorithm. Leetcode 494: Target Sum	24-26
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	27-31
9	Implement Fractional Knapsack using Greedy technique. Leetcode 122: Split a string in balanced strings.	32-35
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm. Leetcode 316: Remove duplicate numbers.	36-39
11	Implement "N-Queens Problem" using Backtracking.	40-42

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.

CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.
-----	---

Lab program 1:

Write program to obtain the Topological ordering of vertices in a given digraph.

CODE:

```
#include <stdio.h>

#define MAX 10

int main() {
    int adj[MAX][MAX], indegree[MAX];
    int queue[MAX], front = 0, rear = 0;
    int n, i, j, count = 0, v;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            scanf("%d", &adj[i][j]);
        }
    }

    for(i = 0; i < n; i++) {
        indegree[i] = 0;
    }

    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            if(adj[i][j] == 1)
                indegree[j]++;
        }
    }

    for(i = 0; i < n; i++) {
        if(indegree[i] == 0)
            queue[rear++] = i;
    }

    printf("Topological Order: ");

    while(front < rear) {
        v = queue[front++];
        printf("%d ", v);
        count++;

        for(j = 0; j < n; j++) {
            if(adj[v][j] == 1) {
                indegree[j]--;
            }
        }
    }
}
```

```

        if(indegree[j] == 0)
            queue[rear++] = j;
    }
}

if(count != n)
    printf("\nGraph contains a cycle. Topological ordering not possible.\n");

return 0;
}

```

OUTPUT:

```

C:\Users\BMSCECSE-L3-22\De
Enter number of vertices: 4
Enter adjacency matrix:
0 1 1 0
0 0 0 1
0 0 0 1
0 0 0 0
Topological Order: 0 1 2 3
Process returned 0 (0x0)   execution time : 21.475 s
Press any key to continue.
|

```

LEETCODE: 207.COURSE SCHEDULE

```
C v Auto
1 bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int* prerequisitesColSize) {
2     int indegree[numCourses];
3     int graph[numCourses][numCourses];
4
5     for (int i = 0; i < numCourses; i++) {
6         indegree[i] = 0;
7         for (int j = 0; j < numCourses; j++) {
8             graph[i][j] = 0;
9         }
10    }
11
12    for (int i = 0; i < prerequisitesSize; i++) {
13        int a = prerequisites[i][0];
14        int b = prerequisites[i][1];
15
16        graph[b][a] = 1;
17        indegree[a]++;
18    }
19
20    int queue[numCourses];
21    int front = 0, rear = 0;
22
23    for (int i = 0; i < numCourses; i++) {
24        if (indegree[i] == 0)
25            queue[rear++] = i;
26    }
27
28    int count = 0;
29
30    while (front < rear) {
31        int curr = queue[front++];
32        count++;
33
34        for (int i = 0; i < numCourses; i++) {
35            if (graph[curr][i]) {
36                indegree[i]--;
37                if (indegree[i] == 0)
38                    queue[rear++] = i;
39            }
40        }
41    }
42
43    return count == numCourses;
44 }
```

OUTPUT:

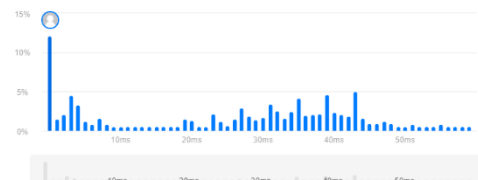
Description Accepted Editorial Solutions Submissions

All Submissions

Accepted 54 / 54 testcases passed
Chandana_J_Kumar submitted at Jun 07, 2025 00:04

Runtime
0 ms | Beats 100.00%

Memory
12.39 MB | Beats 84.50%



Code | C

```
1 #include <stdbool.h>
2 #include <stdlib.h>
3
4 // adjacency list node
5 typedef struct Node {
6     int course;
7     struct Node* next;
8 } Node;
```

View more

Code

C v Auto

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

numCourses =
2

prerequisites =
[[1,0]]

Output

true

Expected

true

Contribute a testcase

Lab Program 2:

Implement Johnson Trotter algorithm to generate permutations.

CODE:

```
#include <stdio.h>

#define LEFT 0
#define RIGHT 1

void printPermutation(int a[], int n) {
    int i;
    for(i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
}

int getMobile(int a[], int dir[], int n) {
    int mobile = 0, mobile_prev = 0;
    int i;

    for(i = 0; i < n; i++) {

        if(dir[a[i]-1] == LEFT && i != 0) {
            if(a[i] > a[i-1] && a[i] > mobile_prev) {
                mobile = a[i];
                mobile_prev = mobile;
            }
        }

        if(dir[a[i]-1] == RIGHT && i != n-1) {
            if(a[i] > a[i+1] && a[i] > mobile_prev) {
                mobile = a[i];
                mobile_prev = mobile;
            }
        }
    }

    return mobile;
}

void johnsonTrotter(int n) {
    int a[n], dir[n];
    int i, j;
```



```

for(i = 0; i < n; i++) {
    a[i] = i + 1;
    dir[i] = LEFT;
}

printPermutation(a, n);

for(i = 1; i < n; i++) {

    int mobile = getMobile(a, dir, n);

    if(mobile == 0)
        return;

    int pos;

    for(j = 0; j < n; j++) {
        if(a[j] == mobile) {
            pos = j;
            break;
        }
    }

    if(dir[mobile-1] == LEFT) {
        int temp = a[pos];
        a[pos] = a[pos-1];
        a[pos-1] = temp;
        pos--;
    }
    else if(dir[mobile-1] == RIGHT) {
        int temp = a[pos];
        a[pos] = a[pos+1];
        a[pos+1] = temp;
        pos++;
    }

    for(j = 0; j < n; j++) {
        if(a[j] > mobile)
            dir[a[j]-1] = !dir[a[j]-1];
    }

    printPermutation(a, n);

    i = 0;
}
}

```

```

int main() {
    int n;
    9 | Page

```

```
printf("Enter value of n: ");  
scanf("%d", &n);  
  
johnsonTrotter(n);  
  
return 0;  
}
```

OUTPUT:

```
Enter value of n: 3  
1 2 3  
1 3 2  
3 1 2  
3 2 1  
2 3 1  
2 1 3  
  
Process returned 0 (0x0)   execution time : 7.742 s  
Press any key to continue.  
|
```

Lab Program 3:

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

CODE:

```
#include <stdio.h>
#include <time.h>

void merge(int a[], int low, int mid, int high)
{
    int i, j, k;
    int b[100];

    i = low;
    j = mid + 1;
    k = low;

    while(i <= mid && j <= high)
    {
        if(a[i] < a[j])
            b[k++] = a[i++];
        else
            b[k++] = a[j++];
    }

    while(i <= mid)
        b[k++] = a[i++];

    while(j <= high)
        b[k++] = a[j++];

    for(i = low; i <= high; i++)
        a[i] = b[i];
}

void mergesort(int a[], int low, int high)
{
    int mid;

    if(low < high)
    {
        mid = (low + high) / 2;

        mergesort(a, low, mid);
        mergesort(a, mid + 1, high);

        merge(a, low, mid, high);
    }
}
```

```
int main()
{
    int a[100], n, i;
    clock_t start, end;
    double time_taken;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter the elements:\n");

    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    start = clock();

    mergesort(a, 0, n - 1);

    end = clock();

    time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("\nSorted Elements are:\n");

    for(i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\n\nTime taken = %f seconds\n", time_taken);

    return 0;
}
```

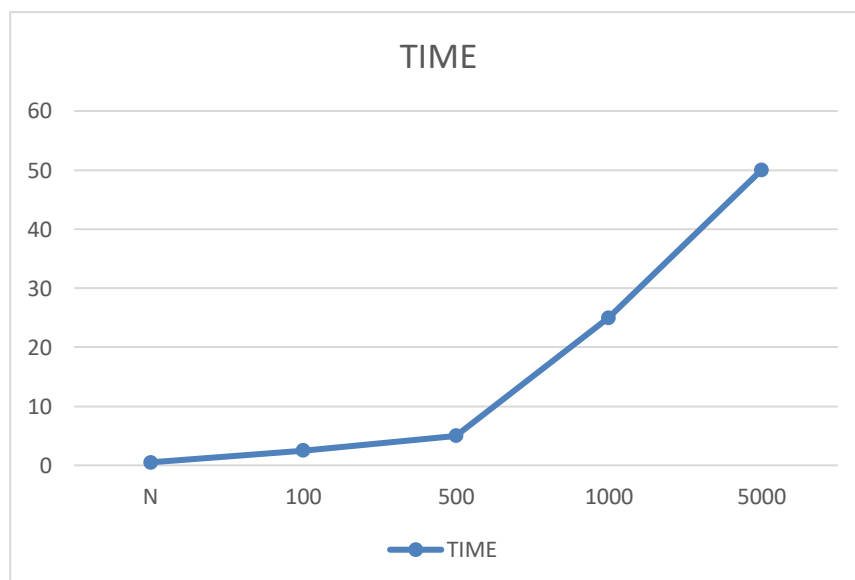
OUTPUT:

```
"C:\Users\BMSCECSE-L3-22\D" X + v
Enter number of elements: 5
Enter the elements:
45 12 89 34 21

Sorted Elements are:
12 21 34 45 89

Time taken = 0.000000 seconds

Process returned 0 (0x0)   execution time : 34.940 s
Press any key to continue.
|
```



Lab Program 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

CODE:

```
#include <stdio.h>
#include <time.h>

void quicksort(int a[], int low, int high);
int partition(int a[], int low, int high);

int main()
{
    int a[100], n, i;
    clock_t start, end;
    double time_taken;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter the elements:\n");

    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    start = clock();

    quicksort(a, 0, n - 1);

    end = clock();

    time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("\nSorted elements are:\n");

    for(i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\n\nTime taken = %f seconds\n", time_taken);

    return 0;
}

void quicksort(int a[], int low, int high)
{
    int j;
```

```

    if(low < high)
    {
        j = partition(a, low, high);

        quicksort(a, low, j - 1);
        quicksort(a, j + 1, high);
    }
}

int partition(int a[], int low, int high)
{
    int pivot, i, j, temp;

    pivot = a[low];
    i = low + 1;
    j = high;

    while(i <= j)
    {
        while(a[i] <= pivot && i <= high)
            i++;

        while(a[j] > pivot)
            j--;

        if(i < j)
        {
            temp = a[i];
            a[i] = a[j];
            a[j] = temp;
        }
    }

    temp = a[low];
    a[low] = a[j];
    a[j] = temp;

    return j;
}

```

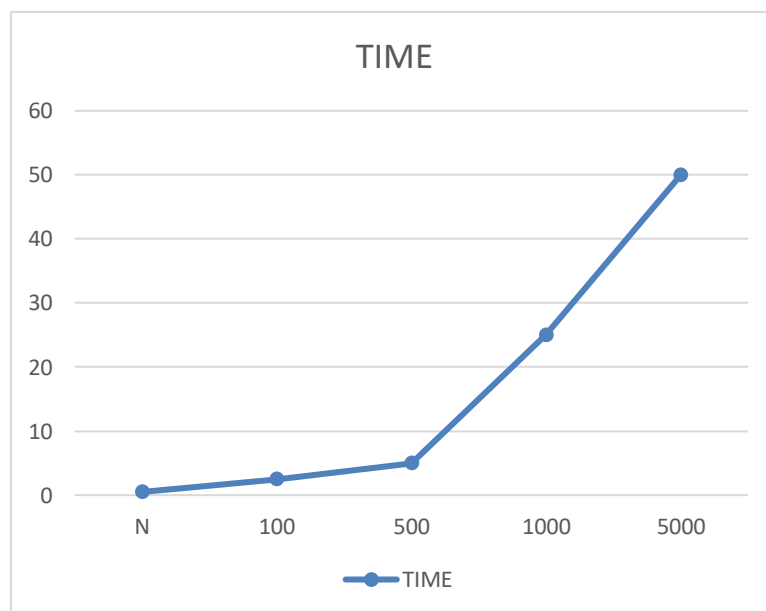
OUTPUT:

```
Enter number of elements: 5
Enter the elements:
45 12 89 34 21

Sorted elements are:
12 21 34 45 89

Time taken = 0.000000 seconds

Process returned 0 (0x0)   execution time : 27.003 s
Press any key to continue.
|
```



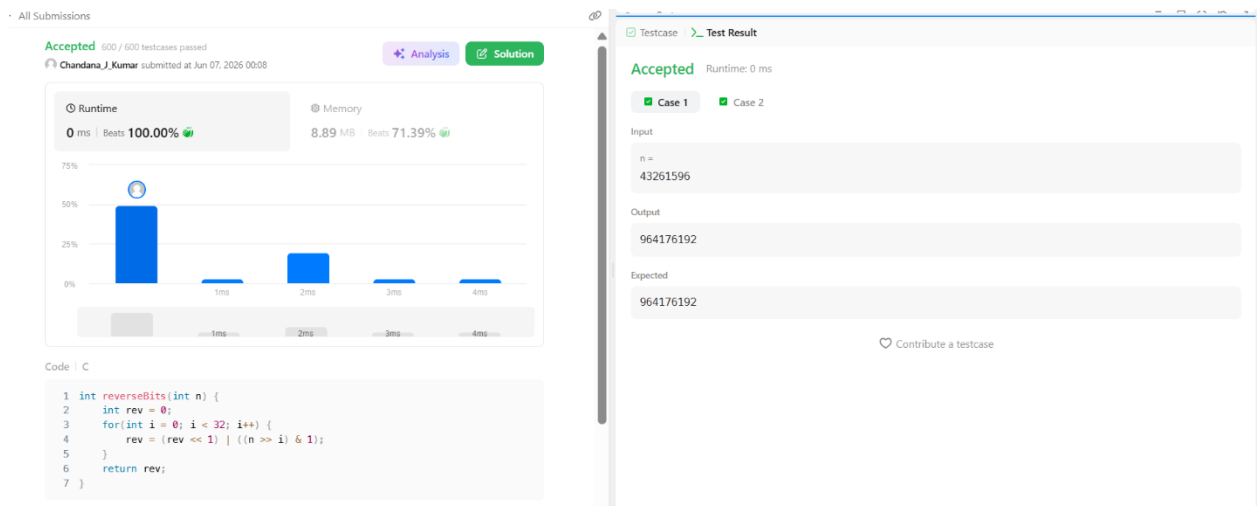
LEETCODE: 190. REVERSE BITS

</> Code

C Auto

```
1 uint32_t reverseBits(uint32_t n) {
2     uint32_t result = 0;
3
4     for (int i = 0; i < 32; i++) {
5         result = (result << 1) | (n & 1);
6         n >>= 1;
7     }
8
9     return result;
10 }
11
```

OUTPUT:



Lab Program 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

CODE:

```
#include <stdio.h>

void heap(int a[], int n, int p)
{
    int c, item;
    item = a[p];
    c = 2 * p + 1;
    while(c < n)
    {
        if(c + 1 < n && a[c] < a[c + 1])
            c++;
        if(item < a[c])
        {
            a[p] = a[c];
            p = c;
            c = 2 * p + 1;
        }
        else
        {
            break;
        }
    }
    a[p] = item;
}

void build_heap(int a[], int n)
{
    int i;
    for(i = (n / 2) - 1; i >= 0; i--)
```

18 | P a g e

```

    {
        heap(a, n, i);
    }
}

void heap_sort(int a[], int n)
{
    int i, temp;
    build_heap(a, n);
    for(i = n - 1; i > 0; i--)
    {
        temp = a[0];
        a[0] = a[i];
        a[i] = temp;
        heap(a, i, 0);
    }
}

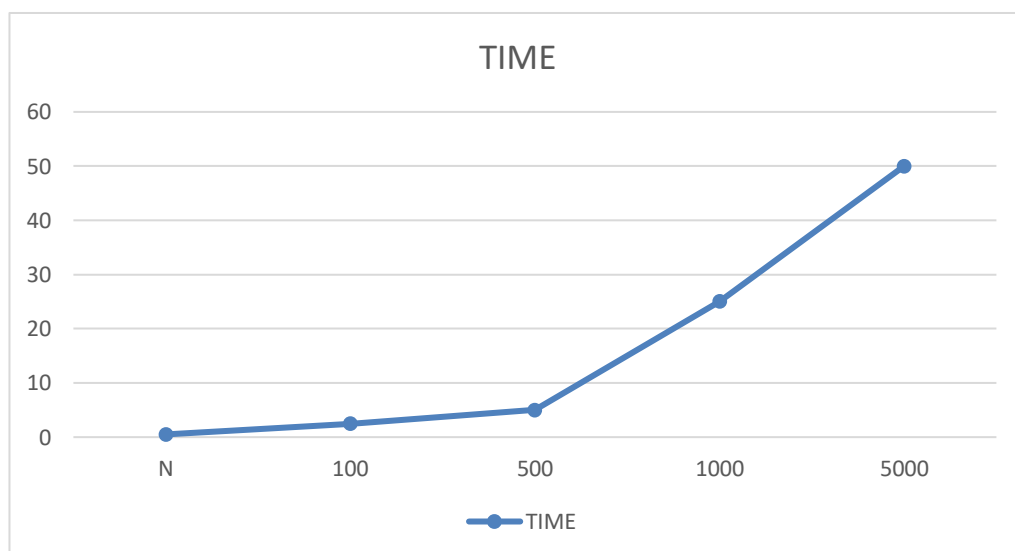
int main()
{
    int a[20], n, i;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }
    heap_sort(a, n);
    printf("Sorted elements:\n");
    for(i = 0; i < n; i++)
    {

```

```
    printf("%d ", a[i]);  
}  
return 0;  
}
```

OUTPUT:

```
C:\Users\GMSCECSE-ES-21\Documents> g++ 1.cpp && ./1.exe  
Enter number of elements: 4  
Enter elements:  
4 2 7 19  
Sorted elements:  
2 4 7 19  
Process returned 0 (0x0)   execution time : 9.127 s  
Press any key to continue.  
|
```



Lab Program 6:

Implement 0/1 Knapsack problem using dynamic programming.

CODE:

```
#include <stdio.h>
int max(int a, int b)
{
    if(a > b)
        return a;
    else
        return b;
}
int main()
{
    int n, i, w;
    int wt[20], val[20];
    int W;
    int K[20][50];

    printf("Enter number of items: ");
    scanf("%d", &n);
    printf("Enter weights:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &wt[i]);
    printf("Enter values:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &val[i]);
    printf("Enter knapsack capacity: ");
    scanf("%d", &W);
    for(i = 0; i <= n; i++)
    {
        for(w = 0; w <= W; w++)
        {
            if(i == 0 || w == 0)
                K[i][w] = 0;
```

```

else if(wt[i-1] <= w)
{
    K[i][w] = max(val[i-1] + K[i-1][w - wt[i-1]],
                  K[i-1][w]);
}
else
{
    K[i][w] = K[i-1][w];
}
}
}
printf("\nMaximum value = %d\n", K[n][W]);
return 0;
}

```

OUTPUT:

```

C:\Users\BMSCECSE-L3-22\De  X  +  v
Enter number of items: 4
Enter weights:
1 2 3 4
Enter values:
1 4 5 7
Enter knapsack capacity: 7

Maximum value = 12

Process returned 0 (0x0)   execution time : 22.932 s
Press any key to continue.
|

```

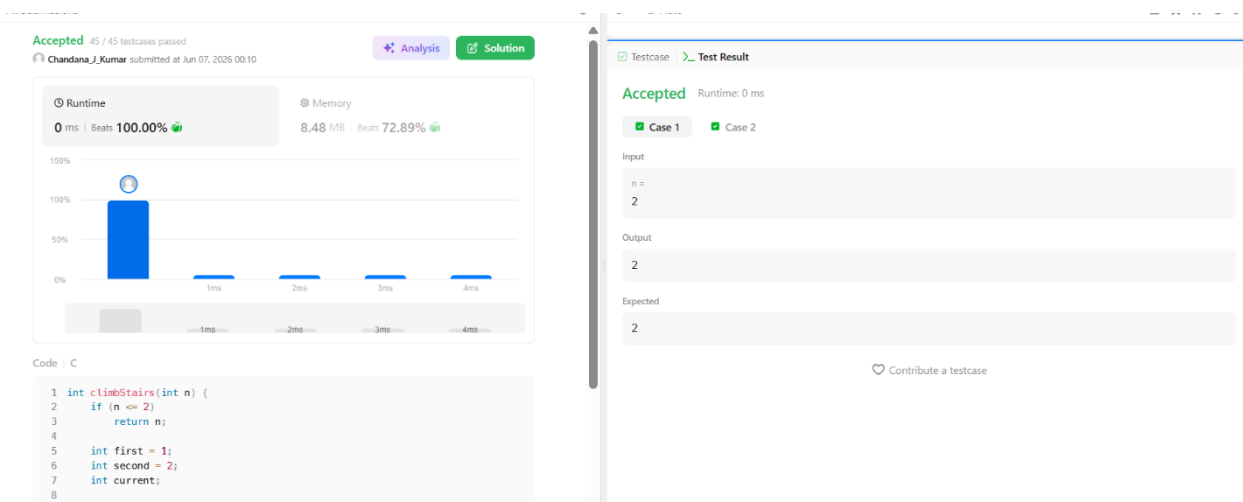
LEETCODE: 70.CLIMBING STAIRS

</> Code

Auto

```
1 int climbStairs(int n) {
2     if (n <= 2)
3         return n;
4
5     int a = 1, b = 2, c;
6
7     for (int i = 3; i <= n; i++) {
8         c = a + b;
9         a = b;
10        b = c;
11    }
12
13    return b;
14 }
```

OUTPUT:



Lab Program 7:

Implement All Pair Shortest paths problem using Floyd's algorithm.

CODE:

```
#include <stdio.h>

#define INF 9999
int main()
{
    int n, i, j, k;
    int dist[10][10];

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter cost matrix (use 0 for no edge):\n");

    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            scanf("%d", &dist[i][j]);

            if (i != j && dist[i][j] == 0)
                dist[i][j] = INF;
        }
    }

    for (k = 0; k < n; k++)
    {
        for (i = 0; i < n; i++)
        {
            for (j = 0; j < n; j++)
            {
                if (dist[i][k] + dist[k][j] < dist[i][j])
                {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }

    printf("\nAll Pair Shortest Path Matrix:\n");

    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            if (dist[i][j] == INF)
                printf("INF\t");
            else
```



```
        printf("%d\t", dist[i][j]);
    }
    printf("\n");
}

return 0;
}
```

OUTPUT:

```
Enter number of vertices: 4
Enter cost matrix (use 0 for no edge):
0 999 6 1
4 0 20 10
999 3 0 12
6 999 999 0

All Pair Shortest Path Matrix:
0      9      6      1
4      0      10     5
7      3      0      8
6      15     12     0

Process returned 0 (0x0)   execution time : 35.585 s
Press any key to continue.
|
```

LEETCODE: 494. TARGET SUM

```
</> Code
C Auto

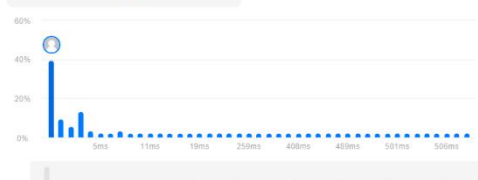
1 int dfs(int* nums, int numsSize, int index, int sum, int target) {
2     if (index == numsSize) {
3         return (sum == target) ? 1 : 0;
4     }
5
6     return dfs(nums, numsSize, index + 1, sum + nums[index], target) +
7         dfs(nums, numsSize, index + 1, sum - nums[index], target);
8 }
9
10 int findTargetSumWays(int* nums, int numsSize, int target) {
11     return dfs(nums, numsSize, 0, 0, target);
12 }
```

OUTPUT:

Accepted 142 / 142 testcases passed
Chandana_J_Kumar submitted at Jun 07, 2025 00:14

Runtime
0 ms | Beats 100.00%

Memory
9.14 MB | Beats 14.32%



Code C

```
1 #include <stdbool.h>
2 #include <stdlib.h>
3
4 int findTargetSumWays(int* nums, int numsSize, int target) {
5     int sum = 0;
6     for (int i = 0; i < numsSize; i++) sum += nums[i];
7
8     // Check feasibility
```

View more

Testcase Test Result

Accepted Runtime: 0 ms

Case 1

Case 2

Input

nums =
[1,1,1,1,1]

target =
3

Output

5

Expected

5

Contribute a testcase

Lab Program 8:

- (a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.
- (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

CODE:

```
#include <stdio.h>
#define INF 9999
int main()
{
    int n, i, j;
    int cost[10][10];
    int visited[10] = {0};
    int edges = 0, mincost = 0;
    int min, a, b;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter cost matrix (enter 0 if no edge):\n");

    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            scanf("%d", &cost[i][j]);

            if (cost[i][j] == 0)
                cost[i][j] = INF;
        }
    }

    visited[0] = 1;

    printf("\nEdges in Minimum Spanning Tree:\n");

    while (edges < n - 1)
    {
        min = INF;

        for (i = 0; i < n; i++)
        {
            if (visited[i])
            {
                for (j = 0; j < n; j++)
                {
```

```

        if (!visited[j] && cost[i][j] < min)
        {
            min = cost[i][j];
            a = i;
            b = j;
        }
    }
}

printf("%d - %d = %d\n", a + 1, b + 1, min);

mincost += min;
visited[b] = 1;
edges++;
}

printf("\nMinimum Cost = %d\n", mincost);

return 0;
}

```

OUTPUT:

```

Enter number of vertices: 4
Enter cost matrix (enter 0 if no edge):
0 10 6 5
10 0 0 5
6 0 0 4
5 15 4 0

Edges in Minimum Spanning Tree:
1 - 4 = 5
4 - 3 = 4
1 - 2 = 10

Minimum Cost = 19

Process returned 0 (0x0)   execution time : 42.803 s
Press any key to continue.
|

```

b)

```
#include <stdio.h>

#define MAX 10

#define INF 999

int main()
{
    int cost[MAX][MAX], dist[MAX], visited[MAX];
    int n, i, j, source, count, mindist, nextnode;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter cost matrix:\n");
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            scanf("%d", &cost[i][j]);
            if (cost[i][j] == 0 && i != j)
                cost[i][j] = INF;
        }
    }

    printf("Enter source vertex: ");
    scanf("%d", &source);

    for (i = 0; i < n; i++)
    {
        dist[i] = cost[source][i];
        visited[i] = 0;
    }

    dist[source] = 0;
    visited[source] = 1;
    count = 1;
```

```

while (count < n - 1)
{
    mindist = INF;
    for (i = 0; i < n; i++)
    {
        if (dist[i] < mindist && !visited[i])
        {
            mindist = dist[i];
            nextnode = i;
        }
    }
    visited[nextnode] = 1;
    for (i = 0; i < n; i++)
    {
        if (!visited[i] &&
            mindist + cost[nextnode][i] < dist[i])
        {
            dist[i] = mindist + cost[nextnode][i];
        }
    }
    count++;
}
printf("\nShortest Paths from Vertex %d:\n", source);
for (i = 0; i < n; i++)
{
    if (i != source)
    {
        printf("%d -> %d = %d\n", source, i, dist[i]);
    }
}

```

```
    return 0;  
}
```

OUTPUT:

```
Enter number of vertices: 4  
Enter cost matrix:  
0   10   6   5  
10  0    0  15  
6   0    0   4  
5   15   4   0  
  
Minimum Spanning Tree:  
1 edge (2,3) = 4  
2 edge (0,3) = 5  
3 edge (0,1) = 10  
  
Minimum Cost = 19  
  
Process returned 0 (0x0)   execution time : 106.393 s  
Press any key to continue.  
|
```

Lab Program 9:

Implement Fractional Knapsack using Greedy technique

CODE:

```
#include <stdio.h>

int main()
{
    int n, i, j;
    float weight[20], profit[20], ratio[20];
    float capacity, temp;
    float totalProfit = 0.0;
    printf("Enter number of items: ");
    scanf("%d", &n);
    printf("Enter weight and profit of each item:\n");
    for (i = 0; i < n; i++)
    {
        scanf("%f %f", &weight[i], &profit[i]);
        ratio[i] = profit[i] / weight[i];
    }
    printf("Enter knapsack capacity: ");
    scanf("%f", &capacity);
    for (i = 0; i < n - 1; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            if (ratio[i] < ratio[j])
            {
                temp = ratio[i];
                ratio[i] = ratio[j];
                ratio[j] = temp;
            }
        }
    }
    for (i = 0; i < n; i++)
    {
        if (weight[i] <= capacity)
        {
            totalProfit += profit[i];
            capacity -= weight[i];
        }
        else
        {
            temp = capacity / weight[i];
            totalProfit += temp * profit[i];
            break;
        }
    }
    printf("Total Profit: %f", totalProfit);
}
```



```

        temp = weight[i];
        weight[i] = weight[j];
        weight[j] = temp;
        temp = profit[i];
        profit[i] = profit[j];
        profit[j] = temp;
    }
}
}
for (i = 0; i < n; i++)
{
    if (weight[i] <= capacity)
    {
        capacity = capacity - weight[i];
        totalProfit = totalProfit + profit[i];
    }
    else
    {
        totalProfit = totalProfit + ratio[i] * capacity;
        break;
    }
}
printf("\nMaximum profit = %.2f\n", totalProfit);
return 0;
}

```

OUTPUT:

```
Enter number of items: 3
Enter weight and profit of each item:
10 60
20 100
30 120
Enter knapsack capacity: 50

Maximum profit = 240.00

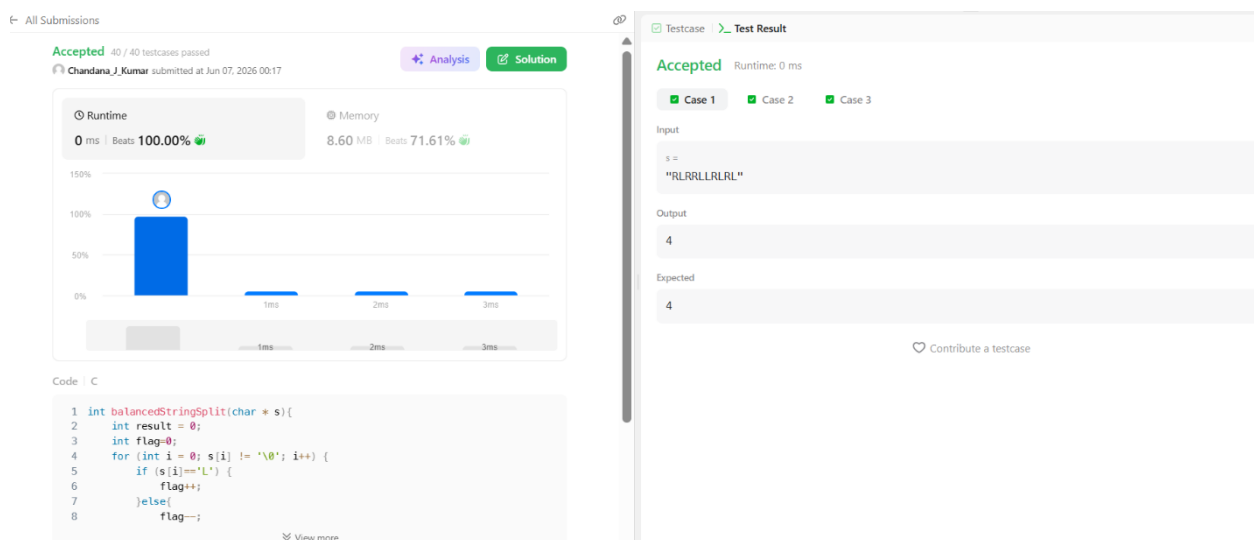
Process returned 0 (0x0)   execution time : 38.167 s
Press any key to continue.
|
```

LEETCODE: 1221. SPLIT A STRING IN BALANCED STRING

C Auto

```
1 int balancedStringSplit(char *s) {
2     int count = 0, balance = 0;
3
4     for (int i = 0; s[i] != '\0'; i++) {
5         if (s[i] == 'R')
6             balance++;
7         else
8             balance--;
9
10        if (balance == 0)
11            count++;
12    }
13
14    return count;
15 }
```

OUTPUT:



Lab Program 10:

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

CODE:

```
#include <stdio.h>

#define MAX 10

#define INF 999

int main()
{
    int cost[MAX][MAX], dist[MAX], visited[MAX];
    int n, i, j, source, count, mindist, nextnode;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter cost matrix:\n");
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            scanf("%d", &cost[i][j]);

            if (cost[i][j] == 0 && i != j)
                cost[i][j] = INF;
        }
    }
    printf("Enter source vertex: ");
    scanf("%d", &source);
    for (i = 0; i < n; i++)
    {
        dist[i] = cost[source][i];
        visited[i] = 0;
```

```

    }
    dist[source] = 0;
    visited[source] = 1;
    count = 1;
    while (count < n - 1)
    {
        mindist = INF;
        for (i = 0; i < n; i++)
        {
            if (dist[i] < mindist && !visited[i])
            {
                mindist = dist[i];
                nextnode = i;
            }
        }
        visited[nextnode] = 1;
        for (i = 0; i < n; i++)
        {
            if (!visited[i] &&
                mindist + cost[nextnode][i] < dist[i])
            {
                dist[i] = mindist + cost[nextnode][i];
            }
        }
        count++;
    }
    printf("\nShortest Paths from Vertex %d:\n", source);
    for (i = 0; i < n; i++)
    {
        if (i != source)

```

```

    {
        printf("%d -> %d = %d\n", source, i, dist[i]);
    }
}
return 0;
}

```

OUTPUT:

```

Enter number of vertices: 5
Enter cost matrix:
0  10  0  30  100
10  0  50  0   0
0  50  0  20  10
100 0  10 60  0
0  100 0  20  30
Enter source vertex: 0

Shortest Paths from Vertex 0:
0 -> 1 = 10
0 -> 2 = 40
0 -> 3 = 30
0 -> 4 = 50

Process returned 0 (0x0)   execution time : 94.692 s
Press any key to continue.
|

```

LEETCODE: 316.REMOVE DUPLICATES LETTERS

```
</> Code
C v Auto

1 char* removeDuplicateLetters(char* s) {
2     int count[26] = {0};
3     int visited[26] = {0};
4     int n = strlen(s);
5
6     for (int i = 0; i < n; i++)
7         count[s[i] - 'a']++;
8
9     char *stack = (char *)malloc((n + 1) * sizeof(char));
10    int top = -1;
11
12    for (int i = 0; i < n; i++) {
13        char c = s[i];
14        count[c - 'a']--;
15
16        if (visited[c - 'a'])
17            continue;
18
19        while (top >= 0 &&
20              stack[top] > c &&
21              count[stack[top] - 'a'] > 0) {
22            visited[stack[top] - 'a'] = 0;
23            top--;
24        }
25
26        stack[++top] = c;
27        visited[c - 'a'] = 1;
28    }
29
30    stack[top + 1] = '\0';
31    return stack;
32 }
```

OUTPUT:

All Submissions

Accepted 290 / 290 testcases passed
Chandana, J. Kumar submitted at Jun 07, 2026 00:20

Analysis Solution

Runtime: 0 ms | Beats 100.00%
Memory: 8.98 MB | Beats 45.56%

Code | C

```
1 #include <stdbool.h>
2 #include <string.h>
3 #include <stdlib.h>
4
5 char* removeDuplicateLetters(char* s) {
6     int n = strlen(s);
7     int count[26] = {0};
8     bool inStack[26] = {false};
```

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

s = "bcabc"

Output

"abc"

Expected

"abc"

Contribute a testcase

Lab Program 11:

Implement “N-Queens Problem” using Backtracking.

CODE:

```
#include <stdio.h>
#include <stdlib.h>
int board[20], count = 0;
int place(int row, int col)
{
    int i;
    for (i = 1; i < row; i++)
    {
        if (board[i] == col || abs(board[i] - col) == abs(i - row))
            return 0;
    }
    return 1;
}
void queen(int row, int n)
{
    int col, i, j;

    for (col = 1; col <= n; col++)
    {
        if (place(row, col))
        {
            board[row] = col;

            if (row == n)
            {
                count++;
                printf("\nSolution %d:\n", count);

                for (i = 1; i <= n; i++)
                {
```



```

        for (j = 1; j <= n; j++)
        {
            if (board[i] == j)
                printf("Q ");
            else
                printf(". ");
        }
        printf("\n");
    }
}
else
{
    queen(row + 1, n);
}
}
}
}

int main()
{
    int n;
    printf("Enter number of queens: ");
    scanf("%d", &n);
    queen(1, n);
    printf("\nTotal solutions = %d\n", count);
    return 0;
}

```

OUTPUT:

```
Enter number of queens: 4

Solution 1:
. Q . .
. . . Q
Q . . .
. . Q .

Solution 2:
. . Q .
Q . . .
. . . Q
. Q . .

Total solutions = 2

Process returned 0 (0x0)   execution time : 2.883 s
Press any key to continue.
|
```